

[Home](#) > [Tutorials](#) > [Machine Learning](#)

Turning Machine Learning Models into APIs in Python

Learn to how to create a simple API from a machine learning model in Python using Flask.

[Contents](#)

Oct 2018 · 20 min read



Sayak Paul

TOPICS

[Machine Learning](#)[Python](#)

Consider the following situation:

You have built a super cool machine learning model that can predict if a particular transaction is fraudulent or not. Now, a friend of yours is developing an android application for general banking activities and wants to integrate your machine learning model in their application for its super objective.

But your friend found out that, you have coded your model in Python while your friend is building his application in Java. So? Won't it be possible to integrate your machine learning model into your friend's application?

Fortunately enough, you have the power of *APIs*. And the above situation is one of the many where the need of turning your machine learning models into APIs is extremely important. Many of the industries are now looking for Data Scientists who can do this. Now, wrapping a machine learning model into an API is not very difficult, and that is precisely what you will be doing in this tutorial - **Turn your machine learning model into an API**.

Options to implement Machine Learning models

Most of the times, the real use of your machine learning model lies at the heart of an intelligent product – that may be a small component of a recommender system or an intelligent chat-bot. These are the times when the barriers seem very difficult to overcome.

For example, the majority of the ML practitioners use R/Python for their experiments. But consumers of those ML models would be software engineers who use a completely different technology stack. There are two ways via which this problem can be solved:

- Rewriting the whole code in the language that the software engineering folks work. The above seems like a good idea, but the time & energy required to get those intricate models replicated would be utterly waste. Majority of languages like JavaScript, do not have great libraries to perform ML. One would be wise to stay away from it.
- API-first approach – Web APIs have made it easy for cross-language applications to work well. If a frontend developer needs to use your ML Model to create an ML powered web application, they would just need to get the URL Endpoint from where the API is being served.

Now, before going any further let's study what really is an API.

What are APIs?

"In simple words, an API is a (hypothetical) contract between 2 softwares saying if the user software provides input in a pre-defined format, the later with extend its functionality and provide the outcome to the user software." - [Analytics Vidhya](#)

You can read the following articles to understand why APIs are a popular choice among developers:

- [History of APIs](#)
- [Introduction to APIs](#)

Essentially, APIs are very much like web applications, but instead of giving you a nicely styled HTML page, APIs tend to return data in a standard data-exchange format such as JSON, XML, etc. Once you a developer has the desired output they can style it whatever the way they want. There are many popular ML APIs as well for example - *IBM Watson's ML API* which is capable of the following:

- Machine Translation - Helps translate text in different language pairs.
- Message Resonance – To find out the popularity of a phrase or word with a predetermined audience.
- Question and Answers - This service provides direct answers to the queries that are triggered by primary document sources.
- User Modelling – To make predictions about social characteristics of someone from a given text.

[Google Vision API](#) is also an excellent example which provides dedicated services for Computer Vision tasks. [Click here](#) to get an idea of what can be done using Google Vision

API.

Basically what happens is a majority of the cloud providers, and smaller machine learning focused companies provide ready-to-use APIs. They cater to the needs of developers/businesses that do not have expertise in ML, who want to implement ML in their processes or product suites.

Popular examples of machine learning APIs suited explicitly for web development stuff are [DialogFlow](#), [Microsoft's Cognitive Toolkit](#), [TensorFlow.js](#), etc.

Now that you have a fair idea of what APIs are, let's see how you can wrap a machine learning model (developed in Python) into an API in Python.

Master your data skills with DataCamp

More than 10 million people learn Python, R, SQL, and other tech skills using our hands-on courses crafted by industry experts.

Start Learning



Flask – A Web Services' Framework in Python:

Now, you might think what is a web service? Web service is a form of API only that assumes that an API is hosted over a server and can be consumed. Web API, Web Service - these terms are generally used interchangeably.

Coming to Flask, it is a web service development framework in Python. It is not the only one in Python, there couple others as well such as Django, Falcon, Hug, etc. But you will use Flask for this tutorial. For learning about Flask, you can refer to [these tutorials](#).

If you downloaded the Anaconda distribution, you already have Flask installed. Otherwise, you will have to install it yourself with:

```
pip install flask
```



Explain code

POWERED BY datacamp

Flask is very minimal. Flask is favorite with Python developers for many reasons. Flask framework comes with an inbuilt light-weighted web server which needs minimal configuration, and it can be controlled from your Python code. This is one of the reasons why it is so popular.

Following code demonstrate Flask's minimality in a nice way. The code is used to create a simple Web-API which upon receiving a particular URL produces a specific output.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "Welcome to machine learning model APIs!"

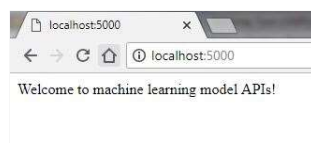
if __name__ == '__main__':
    app.run(debug=True)
```



Explain code

POWERED BY datacamp

Once executed, you can navigate to the web address (enter the address on a Web-Browser), which is shown on the terminal, and observe the result.



Some points:

- Jupyter Notebooks are great for anything related to markdowns, R and Python. But when it comes to building a web server, it may show inconsistent behavior. So, it is a good idea to write the Flask codes in a text editor like *Sublime* and run the code from the terminal/command prompt.
- Make sure you don't name the file as *flask.py*.
- Flask runs on port number 5000 by default. Sometimes, the Flask server starts on this port number successfully, but when you hit the URL (that the servers return on the terminal) in a web browser or any API-client like *Postman*, you may not get the output. Consider the following situation:

```
C:\Users\Sayak\Desktop>python sayak.py
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger pin code: 161-511-689
```

- According to Flask, its server has started successfully on port 5000, but when the URL was fired in the browser, it didn't return anything. So, this can be a possible case of port

number conflict. In this case, changing the default port 5000 to your desired port number would be a good choice. You can do that just by doing the following:

```
app.run(debug=True,port=12345)
```

- In that case, the Flask server would look something like the following:

```
C:\Users\Sayak\Desktop>python sayak.py
* Running on http://127.0.0.1:12345/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger pin code: 161-511-689
```

Now, let's go through step by step of the code that you wrote:

- You created an instance of the `Flask` class and passed in the `"name"` variable (which is filled by Python itself). This variable will be `"main"`, if this file is being directly run through Python as a script. If you imported the file instead, the value of `"name"` would be the name of the file which you imported. For example, if you had `test.py` and `run.py`, and you imported `test.py` into `run.py` the `"name"` value of `test.py` will be `test` (`app = Flask(test)`).
- Above `hello()` method definition, there is `@app.route("")`. `route()` is a **decorator** that tells Flask what URL should trigger the function defined as `hello()`.
- The `hello()` method is responsible for producing an output (Welcome to machine learning model APIs!) whenever your API is properly hit (or consumed). In this case, hitting a web-browser with `localhost:5000/` will produce the intended output (provided the flask server is running on port 5000).

You will now study some of the factors that you will need to keep in mind if you are turning your machine learning models (built using scikit-learn) into a Flask API.

Scikit-learn Models with Flask

Creating very simple to very complex machine learning models have never been this easy in Python with `scikit-learn`. But there are some points you will have to remember about `scikit-learn`:

- **Scikit-learn** is a Python library which provides simple and efficient tools for data mining and data analysis. Scikit-learn has the following major modules:
 - Clustering
 - Regression
 - Classification
 - Dimensionality Reduction
 - Model selection
 - Preprocessing

(Be sure to check DataCamp's [Supervised Learning with scikit-learn](#) course which is taught by the core developer of `scikit-learn` - Andreas Müller)

- Scikit-learn provides the support of serialization and de-serialization of the models that you train using `scikit-learn`. This saves you the time to retrain a model. With a serialized copy of your model made using `scikit-learn` you can write a Flask API.
- Scikit-learn models require the data to be in numerical format. That is why, if the dataset contains categorical features that are non-numeric, it is important to convert them into numeric ones. For this transformation, `scikit-learn` provides utilities like `LabelEncoder`, `OneHotEncoder`, etc. These can be found in `sklearn.preprocessing` module.
- Scikit-learn models cannot handle missing values implicitly. You need to handle missing values in your dataset by yourself, and then you can feed it to your model. For handling missing values, `scikit-learn` provides a wide range of utilities which can be found from `sklearn.preprocessing` module.

Label encoding and missing values are important data preprocessing steps which are very essential for building a good machine learning model. If you want to learn more on this, be sure to check the following course offered by DataCamp:

- [Preprocessing for Machine Learning in Python](#)

For this tutorial, you will use the Titanic dataset which is one of the most popular datasets for many reasons such as - the dataset contains a well great different types of variables, and the dataset contains missing values, etc. This [DataCamp tutorial](#) covers an excellent analysis of the dataset, and the dataset can be downloaded from [here](#).

This dataset deals with a classification problem of predicting if a passenger would survive or not given some information about him/her.

Note: Variables and Features these terms are used interchangeably at many times in this tutorial.

To simplify things even further, you will only use four variables: `age`, `sex`, `embarked`, and `survived` where `survived` is the class label.

```
# Import dependencies
import pandas as pd
import numpy as np
```



Explain code

POWERED BY datalab

```
# Load the dataset in a dataframe object and include only four features as r
url = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/train.csv"
df = pd.read_csv(url)
```



```
include = ['Age', 'Sex', 'Embarked', 'Survived'] # Only four features
df_ = df[include]
```

[🔗 Explain code](#)
POWERED BY  datalab

"Sex" and "Embarked" are categorical features with non-numeric values and that is why they require some numeric transformations. "Age" feature has missing values. These values can be imputed with a summary statistic such as median or mean. Missing values can be quite meaningful, and it is worth investigating what they represent in real-world applications.

Scikit-learn treats the cell values which do not contain anything as NaNs. Here, you will merely replace NaNs with 0, and you will write a helper function for that.

```
categoricals = []
for col, col_type in df_.dtypes.iteritems():
    if col_type == 'O':
        categoricals.append(col)
    else:
        df_[col].fillna(0, inplace=True)
```

[🔗](#)
[🔗 Explain code](#)
POWERED BY  datalab

The above lines of code does the following:

- Iterates over all the columns in the dataframe `df` and appending the columns (with non-numeric values) to a list `categorical`.
- If the columns do not have non-numeric values (which is only `Age` in this case), then it checks if it has missing values or not and fills them with 0.

Filling NaNs with a single value may have unintended consequences, especially if the amount that you're replacing NaNs with is within the observed range for the numeric variable. Since zero is not an observed and legitimate age value you are not introducing bias, you would have if you used say 36! - [Source](#)

Now that you handled the missing values and separated the non-numeric columns you are ready to convert them to numeric ones. You will do this by using [One Hot Encoding](#). Pandas provides a simple method `get_dummies()` for creating OHE variables for a given dataframe.

```
df_ohe = pd.get_dummies(df_, columns=categoricals, dummy_na=True)
```

[🔗](#)
[🔗 Explain code](#)
POWERED BY  datalab

When you use OHE, a new column is created for every column/value combination, in a `column_value` format. For example - for the "Embarked" variable, OHE will produce "Embarked_C", "Embarked_Q", "Embarked_S", and "Embarked_nan".

Now that you've successfully preprocessed your dataset, you're ready to train the machine learning model. You will use a *Logistic Regression classifier* for this.

```
from sklearn.linear_model import LogisticRegression
dependent_variable = 'Survived'
x = df_ohe[df_ohe.columns.difference([dependent_variable])]
y = df_ohe[dependent_variable]
lr = LogisticRegression()
lr.fit(x, y)
```

[🔗](#)
[🔗 Explain code](#)
POWERED BY  datalab

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
intercept_scaling=1, max_iter=100, multi_class='ovr', n_jobs=1,
penalty='l2', random_state=None, solver='liblinear', tol=0.0001,
verbose=0, warm_start=False)
```

[🔗](#)
[🔗 Explain code](#)
POWERED BY  datalab

You have built your machine learning model. You will now save this model. Technically speaking, you will serialize this model. In Python, you call this **Pickling**.

Saving the Model: Serialization and Deserialization

You will use sklearn's `joblib` for this.

```
from sklearn.externals import joblib
joblib.dump(lr, 'model.pkl')
```

[🔗](#)
[🔗 Explain code](#)
POWERED BY  datalab

```
['model.pkl']
```

[🔗](#)
[🔗 Explain code](#)
POWERED BY  datalab

The Logistic Regression model is now persisted. You can load this model into memory with a single line of code. Loading the model back into your workspace is known as Deserialization.

```
lr = joblib.load('model.pkl')
```

[🔗](#)
[🔗 Explain code](#)
POWERED BY  datalab

You're now ready to use Flask to serve your persisted model. You have already seen how minimalistic Flask is to get started with.

Creating an API From a Machine Learning Model using Flask

For serving your model with Flask, you will do the following two things:

- Load the already persisted model into memory when the application starts,
- Create an API endpoint that takes input variables, transforms them into the appropriate format, and returns predictions.

More specifically, your sample input to the API will look like the following:

```
[
  {"Age": 85, "Sex": "male", "Embarked": "S"},
  {"Age": 24, "Sex": "female", "Embarked": "C"},
  {"Age": 3, "Sex": "male", "Embarked": "C"},
  {"Age": 21, "Sex": "male", "Embarked": "S"}
]
```

[Explain code](#) POWERED BY  datalab

(which is a JSON list of inputs)

and your API will output like the following:

```
{"prediction": [0, 1, 1, 0]}
```

[Explain code](#) POWERED BY  datalab

The predictions denote the survival statuses where 0 represents No and 1 represents Yes.

JSON stands for JavaScript Object Notation, and it is one of the most widely used data interchange formats. If you need a quick introduction to it, please follow [these tutorials](#).

Let's write a function `predict()` which will do:

- Load the persisted model into memory when the application starts,
- Create an API endpoint that takes input variables, transforms them into the appropriate format, and returns predictions.

You have already seen how to load a persisted model. Now, you will focus on how you can use it for predicting the survival status upon receiving inputs.

```
from flask import Flask, jsonify
app = Flask(__name__)
@app.route('/predict', methods=['POST'])
def predict():
    json_ = request.json
    query_df = pd.DataFrame(json_)
    query = pd.get_dummies(query_df)
    prediction = lr.predict(query)
    return jsonify({'prediction': list(prediction)})
```

[Explain code](#) POWERED BY  datalab

Fantastic! But you have got a little problem here.

The function that you wrote would only work under conditions where the incoming request contains all possible values for the categorical variables which may or may not be the case in real-time. If the incoming request does not include all possible values of the categorical variables then as per the current method definition of `predict()`, `get_dummies()` would generate a dataframe that has fewer columns than the classifier expects, which would result in a runtime error.

To solve this problem, you will persist the list of columns during model training as well. You can serialize any Python object into a .pkl file. You will use `joblib` in the same way as previously.

(Keep that in mind, as discussed earlier it is always better to do all the server level coding in a text editor and then run it from a terminal)

```
model_columns = list(x.columns)
joblib.dump(model_columns, 'model_columns.pkl')
```

[Explain code](#) POWERED BY  datalab

```
['model_columns.pkl']
```

[Explain code](#) POWERED BY  datalab

As you have persisted the list of columns already, you can just handle the missing values at the time of prediction. You will have to load model columns when the application starts.

```
@app.route('/predict', methods=['POST']) # Your API endpoint URL would consist of /predict
def predict():
    if lr:
        try:
            json_ = request.json
            query = pd.get_dummies(pd.DataFrame(json_))
            query = query.reindex(columns=model_columns, fill_value=0)
```

```

        prediction = list(lr.predict(query))

        return jsonify({'prediction': prediction})

    except:

        return jsonify({'trace': traceback.format_exc()})

    else:
        print ('Train the model first')
        return ('No model here to use')

```

[🔗 Explain code](#)
POWERED BY  datalab

You included all the required elements in the `/predict` API, and now you just need to write the main class.

```

if __name__ == '__main__':
    try:
        port = int(sys.argv[1]) # This is for a command-line argument
    except:
        port = 12345 # If you don't provide any port then the port will be set to
        lr = joblib.load(model_file_name) # Load "model.pkl"
        print ('Model loaded')
        model_columns = joblib.load(model_columns_file_name) # Load "model_columns.pkl"
        print ('Model columns loaded')
        app.run(port=port, debug=True)

```

[🔗 Explain code](#)
POWERED BY  datalab

Your API now ready to be hosted. But before going any further, let's recap all that you did till this point:

Putting it all together:

- You loaded Titanic dataset and selected the four features.
- You did the necessary data preprocessing.
- You built a Logistic Regression classifier and serialized it.
- You also serialized all the columns from training as a solution to the less than expected number of columns is to persist the list of columns from training.
- You then wrote a simple API using Flask that would predict if a person had survived in the shipwreck given there age, sex and embarked information.

Let's put all the code in one place so that you don't miss out on anything. Also, it is a good programming practice if you separate your Logistic Regression model code and your Flask API code into separate `.py` files.

So your `model.py` should look like the following:

```

# Import dependencies
import pandas as pd
import numpy as np

# Load the dataset in a dataframe object and include only four features as mention
url = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/train.csv"
df = pd.read_csv(url)
include = ['Age', 'Sex', 'Embarked', 'Survived'] # Only four features
df_ = df[include]

# Data Preprocessing
categoricals = []
for col, col_type in df_.dtypes.iteritems():
    if col_type == 'O':
        categoricals.append(col)
    else:
        df_[col].fillna(0, inplace=True)

df_ohe = pd.get_dummies(df_, columns=categoricals, dummy_na=True)

# Logistic Regression classifier
from sklearn.linear_model import LogisticRegression
dependent_variable = 'Survived'
x = df_ohe[df_ohe.columns.difference([dependent_variable])]
y = df_ohe[dependent_variable]
lr = LogisticRegression()
lr.fit(x, y)

# Save your model
from sklearn.externals import joblib
joblib.dump(lr, 'model.pkl')
print("Model dumped!")

# Load the model that you just saved
lr = joblib.load('model.pkl')

# Saving the data columns from training
model_columns = list(x.columns)
joblib.dump(model_columns, 'model_columns.pkl')
print("Models columns dumped!")

```

[🔗 Explain code](#)
POWERED BY  datalab

Your `api.py` should look like the following:

```

# Dependencies
from flask import Flask, request, jsonify
from sklearn.externals import joblib

```

```

import traceback
import pandas as pd
import numpy as np

# Your API definition
app = Flask(__name__)

@app.route('/predict', methods=['POST'])
def predict():
    if lr:
        try:
            json_ = request.json
            print(json_)
            query = pd.get_dummies(pd.DataFrame(json_))
            query = query.reindex(columns=model_columns, fill_value=0)

            prediction = list(lr.predict(query))

            return jsonify({'prediction': str(prediction)})

        except:
            return jsonify({'trace': traceback.format_exc()})

    else:
        print('Train the model first')
        return ('No model here to use')

if __name__ == '__main__':
    try:
        port = int(sys.argv[1]) # This is for a command-line input
    except:
        port = 12345 # If you don't provide any port the port will be set to 1234

    lr = joblib.load("model.pkl") # Load "model.pkl"
    print('Model loaded')
    model_columns = joblib.load("model_columns.pkl") # Load "model_columns.pkl"
    print('Model columns loaded')

    app.run(port=port, debug=True)

```


[Explain code](#)

POWERED BY datalab

Pretty neat! Now you will test this API in an API client called [Postman](#). Just make sure that `model.py` and `api.py` are in the same directory and also make sure that you have compiled them both before testing. Refer to the following snapshot of the terminal which is taken after both the `.py` files were compiled successfully.

```

D:\DataCamp API Blog>python model.py
C:\Users\SARAH\Anaconda3\lib\site-packages\pandas\core\generic.py:5430: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation: http://pandas.pydata.org/pandas-docs/stable/indexing.html#indexing-view-versus-copy
  self._update_inplace(new_data)
Model dumped!
Models columns dumped!

D:\DataCamp API Blog>python api.py 12345
Model loaded
Model columns loaded
* Serving Flask app "api" (lazy loading)
* Environment: production
   WARNING: Do not use the development server in a production environment.
   Use a production WSGI server instead.
* Debug mode: on
* Restarting with stat
Model loaded
Model columns loaded
* Debugger is active!
* Debugger PIN: 115-988-303
* Running on http://127.0.0.1:12345/ (Press CTRL+C to quit)

```

If all of your files were compiled successfully, the following should be the directory structure:

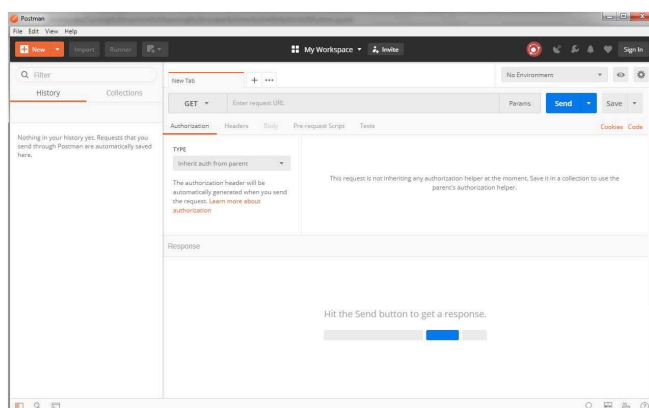
	api.py	9/18/2018 1:05 PM	PY File	2 KB
	model.pkl	9/18/2018 1:16 PM	PKL File	1 KB
	model.py	9/18/2018 1:08 PM	PY File	2 KB
	model_columns.pkl	9/18/2018 1:16 PM	PKL File	1 KB
	Turning machine learning models into A...	9/26/2018 5:34 PM	IPYNB File	34 KB

Note: The IPYNB file is optional though.

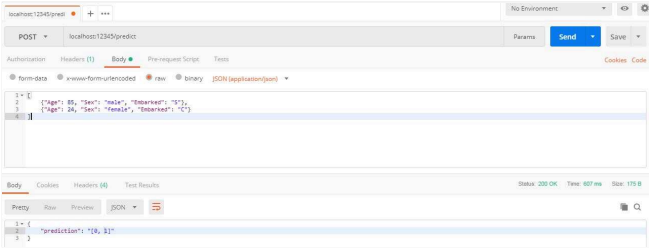
Testing your API in Postman

In order to test your API, you will need some kind of API client. Postman is undoubtedly one of the best ones out there. You can easily download Postman from the above link.

The Postman interface looks like the following if you downloaded the latest one:



After you have started the Flask server successfully, you then need to enter the right URL with the correct port number in Postman. It should look similar to the following:



Congratulations! You just built your first ever machine learning API.

Your API can predict if a passenger survived the Titanic shipwreck given there age , sex and embarked information. Now, your friend may call it from there front-end code and process the output of the API into something fascinating.

Taking it Further:

In this tutorial, you covered one of the most vital industry demanding skills of a full-stack Data Scientist, i.e. building an API from a machine learning model. Although the API is straightforward, it is always better to start with the simplest of things so that you know the know-how in the details.

You can do a lot more in order to improve this. Possible options you might want to consider:

- Write a "/train" API which would train a Logistic Regression classifier with the data.
- Code a Neural Network model using keras and build an API out of it.
- Host your API on Cloud so that it can be consumed.
- For taking things to more advanced levels, you might refer to [this Machine Learning Mastery blog](#) which discusses several industry graded approaches.

The possibilities and opportunities are enormous here. You just need to carefully select the ones which are the most suitable for you.

If you would like to learn more about Machine Learning in Python, take DataCamp's [Preprocessing for Machine Learning in Python](#) course and check out our [Machine Learning Basics - The Norms](#) tutorial.

References:

The following are some references that were taken while writing this blog:

- [Flask: Building Python Web Services](#)
- [A Quora thread on the topic](#)

TOPICS

Machine Learning Python

Learn more about Python and Machine Learning

COURSE

Preprocessing for Machine Learning in Python

🕒 4 hr 🏆 45K

Learn how to clean and prepare your data for machine learning!

See Details →

Start Course

COURSE

Feature Engin

🕒 4 hr 🏆 28.7K

Create new fea

See Details →

See More →

Related

- TUTORIAL
Automated Machine Learning
with Auto-Keras
- TUTORIAL
Python Machine Learning:
Scikit-Learn Tutorial
- TUTORIAL
TensorFlow Tutorial For
Beginners

See More →